

Laboratory 09

Producer-Consumer IPC with FreeRTOS Queue

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXXN236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Estimated time	4–5 hours
CLO mapping	CLO 5 & CLO 6

1. Objectives

By completing this lab you will be able to:

1. **Implement** a high-rate producer task that writes samples into a lock-free ring buffer protected by `__DMB()` memory barriers.
2. **Implement** a consumer task that drains the ring buffer and applies a 4-tap moving-average FIR filter to each sample.
3. **Use** a FreeRTOS queue as a lightweight IPC notification channel between the producer and consumer.
4. **Measure** end-to-end IPC latency using DWT cycle timestamps.
5. **Understand** the design differences between FreeRTOS queue IPC and hardware Messaging Unit (MU) IPC on true dual-core MCUs.

Note

Single-core adaptation. The MCXXN236 is a *single-core* Cortex-M33 device (the “N2” in the part number refers to the MCX N-series family, not core count). This lab implements the same producer-consumer pattern and ring-buffer IPC concepts using **FreeRTOS tasks and queues** on a single core. The theoretical dual-core/MU content in the Background section shows how the design would extend to a true dual-core device (e.g. MCXXN947).

2. Prerequisites

- ☐ Lab 06 complete (DWT cycle counter working).
- ☐ ARM GNU Toolchain 14.2 and pyocd working.
- ☐ NXP MCUXpresso SDK at `C:/nxp_sdk/mcuxsdk/mcuxsdk`.
- ☐ Understanding of memory barriers (`__DMB()`, `__DSB()`).

3. Background

3.1. FreeRTOS Queue as IPC Notification

On a single-core MCU, tasks communicate through FreeRTOS primitives. This lab uses a **queue of depth 1** as a notification channel — the producer sends a token when new samples are available; the consumer blocks on the queue:

```

1  /* Producer sends notification */
2  uint32_t token = 1;
3  xQueueSend(g_notify_queue, &token, 0);    /* non-blocking; drop if
      full */
4
5  /* Consumer blocks for notification */

```

```

6 uint32_t dummy;
7 xQueueReceive(g_notify_queue, &dummy, portMAX_DELAY);

```

3.2. Ring Buffer with Memory Barriers

Even on a single core, `__DMB()` is the correct idiom for lock-free ring buffers because it prevents the compiler and CPU pipeline from reordering the data store before the index update:

```

1 g_ring.data[g_ring.write_idx] = sample;
2 __DMB();                      /* data visible before index advance */
3 g_ring.write_idx = next;

```

3.3. Conceptual Dual-Core Reference (MCXN947 / True AMP)

On a true dual-core device the IPC would use the Messaging Unit (MU) hardware instead of a FreeRTOS queue, and the ring buffer would live in shared SRAM:

```

1 /* Producer -- on CM33_0 (dual-core device only) */
2 g_ring.data[g_ring.write_idx] = sample;
3 __DMB();                      /* all stores visible before index
   update */
4 g_ring.write_idx = next;
5 __DMB();                      /* index visible before MU send */
6 MU_SendMsg(MUA, 0, next);     /* hardware notification to CM33_1
   */
7
8 /* Consumer ISR -- on CM33_1 (dual-core device only) */
9 uint32_t idx = MU_ReceiveMsg(MUB, 0);
10 __DMB();                      /* MU read complete before we read
   shared buffer */
11 process(g_ring.data[idx]);

```

The `__DMB()` requirements are *stricter* on a multi-core device because the memory model is weakly ordered across cores.

4. Project Structure

```

lab09_freertos_ipc/
  cm33_0/                                <- Main (functional) firmware
    Makefile
    src/
      main_cm33_0.c                      <- hardware init, queue + task creation
      adc_producer.c/.h                  <- producer task (simulated ADC at 1 kHz)
      fir_consumer.c/.h                  <- consumer task (4-tap FIR filter)
      shared_mem.h                      <- RingBuf_t, LatencyTest_t types
      FreeRTOSConfig.h
  cm33_1/                                <- Reference build (conceptual; links
    cleanly)
    Makefile
    src/
      main_cm33_1.c                      <- standalone reference entry point
      fir_consumer.c/.h                  <- same consumer logic
      shared_mem.h

```

Tip

Only `cm33_0/` is needed to run the lab on the FRDM-MCXN236. `cm33_1/` is a companion reference build to illustrate how the consumer would be structured on a separate core.

5. Part A — Build and Flash

5.1. Step 1 — Build cm33_0

```
cd labs/lab09_freertos_ipc/cm33_0
mingw32-make
```

Expected output:

text	data	bss	dec	hex	filename
28896	20	69852	98768	181d0	lab09_cm33_0.elf

5.2. Step 2 — Flash

```
mingw32-make flash
```

5.3. Step 3 — Expected UART Output

```
=== RTOS Lab 09: Dual-Core AMP (CM33_0) ===
[CM33_0] starting scheduler -- free heap: 61440 bytes

[CM33_1] consumed=1000, missed=0, free heap=62440
[CM33_1] consumed=2000, missed=0, free heap=62440
```

✓ Checkpoint A

UART showing non-zero consumed count and zero missed samples for at least 10 seconds.

6. Part B — Producer Task

The producer runs at 1 kHz using vTaskDelayUntil. Examine src/adc_producer.c:

```
1 void vADCProducerTask(void *pvParameters)
2 {
3     TickType_t xLastWake = xTaskGetTickCount();
4     uint32_t sample_count = 0;
5
6     for (;;) {
7         vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(1)); /* 1 kHz */
8
9         uint32_t next = (g_ring.write_idx + 1U) % RING_BUF_SIZE;
10        if (next == g_ring.read_idx) {
11            g_ring.missed++;
12            continue; /* ring full -- drop sample */
13        }
14
15        g_ring.data[g_ring.write_idx] = (int16_t)(sample_count %
16            4096);
17        sample_count++;
18        __DMB(); /* data store before index
19            advance */
20        g_ring.write_idx = next;
21
22        /* Notify consumer every 16 samples */
23        if ((g_ring.write_idx & 0xFU) == 0U) {
24            uint32_t token = 1U;
25            xQueueSend(g_notify_queue, &token, 0);
26        }
27    }
```

? Question B1

Why does the producer call `xQueueSend` only every 16 samples rather than every sample? What is the trade-off between notification frequency and latency?

? Question B2

The producer uses `xQueueSend` with timeout 0 (non-blocking). What happens to a notification if the consumer has not yet read the previous one from the queue? Is this safe for a ring-buffer IPC scheme? Why?

7. Part C — Consumer Task and FIR Filter

Examine `src/fir_consumer.c`:

```

1 void vFIRConsumerTask(void *pvParameters)
2 {
3     uint32_t consumed = 0;
4     TickType_t rep_tick = xTaskGetTickCount();
5
6     for (;;) {
7         uint32_t dummy;
8         xQueueReceive(g_notify_queue, &dummy, pdMS_TO_TICKS(100));
9
10        __DMB(); /* ensure latest write_idx is visible */
11
12        while (g_ring.read_idx != g_ring.write_idx) {
13            int16_t raw = g_ring.data[g_ring.read_idx];
14            int32_t filtered = fir_filter(raw); /* 4-tap moving
15            average */
16            g_ring.read_idx = (g_ring.read_idx + 1U) %
17            RING_BUF_SIZE;
18            consumed++;
19            (void)filtered;
20        }
21
22        /* Report every 1 s */
23        if ((xTaskGetTickCount() - rep_tick) >= pdMS_TO_TICKS(1000))
24        {
25            rep_tick = xTaskGetTickCount();
26            PRINTF("[CM33_1] consumed=%lu missed=%lu heap=%u\r\n",
27            (unsigned long)consumed,
28            (unsigned long)g_ring.missed,
29            (unsigned)xPortGetFreeHeapSize());
30        }
31    }
32 }

```

7.1. FIR Filter

The 4-tap moving-average filter in the consumer:

```

1 static int32_t fir_filter(int16_t new_sample)
2 {
3     static int16_t history[4] = {0};
4     static uint8_t idx = 0;
5
6     history[idx] = new_sample;
7     idx = (uint8_t)((idx + 1U) % 4U);

```

```

8
9     int32_t acc = 0;
10    for (uint8_t i = 0; i < 4; i++) acc += history[i];
11    return acc / 4;
12 }

```

✓ Checkpoint C

Modify `fir_consumer.c` to PRINTF the running average every 1000 samples. Show terminal output with filtered values.

? Question C1

The ring buffer uses `volatile` on `write_idx` and `read_idx`, *and* also uses `__DMB()`. What does `volatile` prevent that `__DMB()` does not? What does `__DMB()` prevent that `volatile` does not?

8. Part D — IPC Latency Measurement

Use the DWT cycle counter to measure the latency from when the producer writes the last sample in a batch to when the consumer first reads it. The `shared_mem.h` already defines `LatencyTest_t` for this:

```

1  /* shared_mem.h */
2  typedef struct {
3      volatile uint32_t send_cycles;    /* producer records DWT->CYCCNT
4          before notify */
5      volatile uint32_t recv_cycles;    /* consumer records DWT->CYCCNT
6          on wake-up */
7  } LatencyTest_t;
8
9  extern LatencyTest_t g_latency;

```

Add the measurements:

```

1  /* In vADCProducerTask -- just before xQueueSend */
2  g_latency.send_cycles = DWT->CYCCNT;
3
4  /* In vFIRConsumerTask -- first line after xQueueReceive returns */
5  g_latency.recv_cycles = DWT->CYCCNT;

```

Then print the latency in the consumer's periodic report:

```

1  uint32_t lat = g_latency.recv_cycles - g_latency.send_cycles;
2  PRINTF("[LAT] %lu cycles (~%lu ns @ 150 MHz)\r\n",
3          (unsigned long)lat, (unsigned long)(lat * 1000UL / 150UL));

```

✓ Checkpoint D

Terminal output showing measured IPC latency in cycles and nanoseconds.

? Question D1

On a true dual-core device, MU hardware latency is typically 50–200 cycles. How does your FreeRTOS queue IPC latency compare? What accounts for the difference (FreeRTOS context

switch overhead, queue ISR path, etc.)?

9. Deliverables

#	Item	Details
1	Part A UART	Non-zero consumed, zero missed for 10 s
2	Part C UART	Running average printed every 1000 samples
3	Part D UART	IPC latency in cycles and nanoseconds
4	Written answers	Questions B1, B2, C1, D1
5	<code>shared_mem.h</code> listing	Ring buffer + latency struct

10. Key Concepts Demonstrated

Concept	Where you saw it
Lock-free ring buffer with DMB barriers	Parts B & C
FreeRTOS queue as IPC notification	Parts B & C
IPC latency measurement with DWT	Part D
Producer notification batching	Part B (Q B1)
<code>volatile</code> vs. <code>__DMB()</code>	Part C (Q C1)
MU hardware IPC (conceptual reference)	Background

11. Reference

Resource	Relevant sections
ARM Cortex-M33 TRM	§A3.4 Memory ordering, DMB instruction
Barry, <i>Mastering the FreeRTOS Kernel</i>	Ch. 9 (task notifications), <code>xQueueSend</code> , <code>xQueueReceive</code>
NXP MCUXpresso SDK	<code>fs1_mu.h</code> (for dual-core reference)
NXP MCXN236 Data Sheet	§1 (single-core Cortex-M33 architecture)